

MSDS 535

Week 10 – Chapter 10

Overview and Objectives

- Understand Deep Learning Basic concepts.
- Improve training of deep learning models.
- Convolutional neural networks.
- Recurrent neural networks.
- Graph neural networks.
- Summarize deep learning basic concepts.

What is deep learning?

- Deep learning is based on artificial neural networks (ANNs), also known as neural networks.
- Neural networks consist of connected input-output units with associated weights.
- During the learning phase, weights are adjusted to predict target values, such as class labels.

What is deep learning?

- A unit in a neural network is a mathematical function that takes a linear weighted sum of inputs and passes it through an activation function.
- Activation functions are typically nonlinear, such as the sign function in perceptron or sigmoid function in logistic regression.
- Units in the input layer are called input units, and those in hidden and output layers are referred to as neurodes or neurons.

What is deep learning?

- Multilayer feed-forward neural networks consist of input, hidden, and output layers.
- Multilayer feed-forward networks can model class prediction as a nonlinear combination of inputs.
- They have the ability to learn more complex features from simpler ones as information flows through layers.

What is deep learning?

- Deep neural networks, with many hidden layers, can represent features at different abstraction levels.
- The backpropagation algorithm is used to train deep neural networks.
- Challenges in training deep neural networks are addressed in Section 10.2.
- Deep learning is related to representation learning, which involves learning effective features from input data.

Backpropagation algorithm

- Backpropagation is a foundational technique for learning the parameters (weights and biases) of a neural network.
- It iteratively processes a training dataset, comparing the network's predictions with the known target values.
- The error between the predictions and targets is used to update the weights and biases.

Backpropagation algorithm

- Backpropagation involves several steps, including initializing weights and biases, propagating inputs forward, and backpropagating the error.
- The mean-squared loss is typically used to measure the disagreement between predicted and actual target values.
- Weights are updated using gradient descent, and the learning rate (η) helps control the update step.

Backpropagation algorithm

- Biases are also updated in a similar manner.
- Training stops when specified conditions are met, such as small weight changes, low misclassification rate, or a set number of epochs.
- The number of iterations (epochs) required for convergence can vary widely, and several techniques exist to speed up training.

Backpropagation algorithm

- To classify an unknown tuple, it is fed into the trained network, and the output node with the highest value (Maxout) or the SoftMax function is used to determine the predicted class label.
- Deep learning models can be represented in matrix form, where each layer involves matrix multiplication and element-wise activation functions.
- Training deep learning models often requires the use of GPUs for their computational efficiency and memory access advantages.

Key challenges for training deep learning models

- Backpropagation is a core method for training neural networks.
- It involves fine-tuning the network's internal settings (weights and biases) to improve its performance on a specific task.
- Think of it like adjusting a robot's parts to make it work better.
- Training data, consisting of input-output pairs, is used to teach the network.
- We compare the network's predictions to the correct answers and measure the error.

Key challenges for training deep learning models

- Then, we adjust the network's settings (weights and biases) to minimize this error, making its predictions more accurate.
- However, this process faces two challenges:
 - Optimization challenge: Finding the best settings for the network is like navigating a landscape with hills and valleys.
 - Overfitting challenge: We need to ensure the network learns the task's underlying principles, not just memorizes specific examples.
- Techniques are used to address these challenges and make backpropagation more effective.

Overview of deep learning architecture

- L-layer feed-forward neural networks are mainly for multidimensional data, represented as n_0 -dimensional attribute vectors.
- Different deep neural network types are designed for various data types:
 - Convolutional Neural Networks (CNNs) for grid-like data.
 - Recurrent Neural Networks (RNNs) for sequences.
 - Graph Neural Networks (GNNs) for relational or graph data.
- Each of these architectures is suited for specific data structures and will be discussed in upcoming sections (CNNs, RNNs, GNNs).

Responsive activation functions

- Backpropagation typically uses a function called "sigmoid" to make adjustments in neural networks.
- If the "sigmoid" function gives an output close to 0 or 1, it can cause problems in the learning process.
- When these problems occur, it leads to the weights and biases not changing much, and the learning process stagnates.

Responsive activation functions

- This problem becomes worse when errors are passed from the output to the hidden layers.
- An effective solution to this issue is to use a different function called "ReLU" that avoids these problems.
- With ReLU, inactive units stay inactive, and the parameters don't change much, but active units can adjust more effectively.
- There are also other functions like ReLU that work better than the sigmoid function.

Adaptive learning rate

- Learning rate (η) in backpropagation affects how fast the model parameters change.
- If η is too small, it may take many epochs for the algorithm to converge.
- If η is too large, it can lead to issues like overshooting the desired solution.
- Adaptive learning rate (η_t) that changes with the epoch number can be a more reasonable choice.

Adaptive learning rate

- A common strategy is to start with a larger η and gradually decrease it as the algorithm progresses.
- One simple approach sets η_t in reverse proportion to the epoch number (e.g., $\eta_t = 1/t * \eta_0$).
- After a certain number of epochs, the learning rate can be set to a small constant (η^∞) to prevent it from getting too small.
- AdaGrad and RMSProp are methods that adapt the learning rate based on the magnitude of the gradient.

Dropout

- Deep neural networks can learn complex hierarchies of features but are susceptible to overfitting.
- Overfitting can lead to the learning of fictitious features that perform well on training data but not on test data.
- Dropout is a strategy to prevent overfitting in deep neural networks.
- Dropout randomly deactivates some non-output units in a neural network during each epoch.

Dropout

- Forward and backward propagation occur on this modified "dropout network" to update model parameters.
- Deactivated units are temporarily "frozen" and do not impact parameter updates.
- The idea is to improve robustness and generalization of the network by introducing randomness.
- In practice, a binary mask gate is used to control unit dropout, and a dropout rate $(1 - q)$ is set.

Dropout

- Scaling of model parameters is often performed after training, ensuring similar behavior during testing.
- Dropout acts as a form of regularization and helps networks generalize better to test data.
- Dropout can be seen as analogous to ensemble methods in terms of improving model performance through aggregation.

Pretraining

- Deep learning models aim to minimize nonconvex objective functions.
- The initial model parameters (θ_0) significantly affect the solution quality due to nonconvexity.
- Pretraining initializes model parameters (weights and biases) in a suitable region.
- Greedy supervised pretraining is an effective approach, where simpler models are trained first and used as hints for the complex model.

Pretraining

- Pretraining occurs layer-by-layer in an iterative manner.
- At each iteration, a simple model is trained, incorporating a new hidden layer while keeping parameters from previous iterations fixed.
- This process continues until all hidden layers are added, producing a pretrained model.
- After pretraining, a fine-tuning phase follows to further optimize model parameters.

Pretraining

- Pretraining can be done in supervised, unsupervised, or hybrid settings.
- Transfer learning uses pretrained models to improve performance on a new task.
- In transfer learning, a source network trains on a source task, and its parameters are used to initialize a target network for a related task, followed by fine-tuning.

Cross-entropy

- Backpropagation algorithm employs loss functions to measure discrepancies between actual and predicted target values.
- Cross-entropy is a common choice for binary classification when dealing with a single output unit and sigmoid activation function.
- It prevents gradient vanishing when output unit saturates.
- In multiclass classification, cross-entropy handles C class labels and predicted target values effectively.

Autoencoder: unsupervised deep learning

- Autoencoders are used in scenarios without supervision.
- Autoencoders reconstruct input data, using encoder and decoder.
- Autoencoders can be single-layered (encoder and decoder) or stacked with multiple layers.
- To prevent autoencoders from copying input, constraints, regularization, or denoising training can be applied.

Autoencoder: unsupervised deep learning

- Autoencoders can be employed for unsupervised learning in various applications.
- They reduce dimensionality, similar to PCA, but can handle non-linear data transformations.
- Autoencoders can perform deep clustering by embedding dimensionality reduction and clustering, increasing the clustering quality.

Autoencoder: unsupervised deep learning

- They integrate dimensionality reduction and clustering steps using iterative algorithms.
- Deep clustering supports different architectures and loss functions for specific data types.
- Autoencoders are also used for unsupervised pretraining of neural networks by initializing layers in a greedy way.

Other techniques

- Functions like ReLU enable gradient flow, mitigating vanishing gradient issues during backpropagation.
- Techniques like AdaGrad or RMSProp automatically adjust the learning rate to prevent gradient explosion when gradients are large.
- Dropout is a regularization method that randomly deactivates neurons during training to combat overfitting and enhance generalization.

Other techniques

- Pretraining initializes model parameters, and greedy supervised pretraining is a layer-wise approach for faster convergence.
- Cross-Entropy Loss is useful for classification tasks, addressing gradient vanishing problems with saturated activation functions.
- Autoencoders serve various purposes in unsupervised learning, including dimensionality reduction, deep clustering, and unsupervised pretraining.

Other techniques

- Techniques such as adaptive learning rates and gradient clipping help address gradient explosion problems.
- Early stopping prevents overfitting by monitoring validation errors and terminating training if no improvements occur.
- Batch normalization involves z-score normalization, stabilizing training and alleviating internal covariate shift.

Other techniques

- Moment-based approaches like Adam update parameters considering historical gradients, reducing gradient variance in mini-batches.
- Shortcut connections (ResNets and HighwayNets) facilitate gradient flow by providing direct links between lower and nonadjacent higher layers, mitigating gradient vanishing issues.

Introducing convolution operation

- Smoothing the input time series by averaging values in the local vicinity is useful for identifying the overall trend, while ignoring noise or small fluctuations.
- Detecting sudden changes in the time series requires computing the difference of values before and after certain time stamps.
- Convolution is a powerful technique to automate the extraction of features such as average and difference from input data.
- The convolution operation combines input data and a kernel (filter) to generate a feature map.

Introducing convolution operation

- Each element in the feature map represents a feature corresponding to an element in the input, computed using linear combinations of nearby elements.
- Convolution with a 1-D input, represented as a one-dimensional array, involves a kernel of length P , typically smaller than the input size ($P \ll N$).
- Zero padding can be used to address boundary issues when computing the first and last elements of the feature map.

Introducing convolution operation

- The stride parameter controls the movement of the kernel along the input during convolution.
- Convolution is a fundamental operation in convolutional neural networks (CNNs) used for tasks like image recognition.
- Different kernels allow convolution to function as a smoothing operator or a change (difference) detector, providing flexibility for various feature extraction purposes.

Multidimensional convolution

- Convolution is an operation used to process image and signal data.
- It involves sliding a smaller matrix, called a kernel or filter, over a larger input matrix to produce a feature map.
- Convolution is used in 2-D and 3-D data processing, where the input and kernel are 2-D and 3-D structures, respectively.

Multidimensional convolution

- Zero-padding can be applied to address boundary issues when using convolution.
- One of the advantages of convolution is its computational efficiency due to parameter sharing.
- Convolutional neural networks (CNNs) heavily rely on the convolution operation, making it essential in computer vision and other applications.

Convolutional layer

- In a convolutional layer, multiple kernels are organized into a tensor.
- Each kernel is applied separately to the input volume to perform a convolution operation.
- Each kernel is full in depth, producing 2-D feature maps, and applying all kernels generates multiple feature maps.

Convolutional layer

- Feature maps are stacked into a 3-D tensor with the same depth as the number of kernels.
- Feature map tensors undergo a nonlinear activation step, typically using ReLU to make all negative values zero.
- A pooling operation, such as max pooling or average pooling, reduces the size of feature map tensors.

Convolutional layer

- Pooling helps prevent overfitting and provides approximate translation invariance.
- Convolutional Neural Networks (CNNs) have convolutional layers, often organized into a deep architecture.
- Depth in CNNs is critical for learning complex representations and improving accuracy in image recognition tasks.

Recurrent neural networks

- RNNs are used for sequential data like text, audio, and time series, with applications in translation, speech, and more.
- Input sequences (e.g., words in a sentence or time-stamped data) require modeling of sequential dependencies.
- Techniques like gated RNNs and attention mechanisms are used to handle long-term dependencies effectively.

Basic RNN models and applications

- Sequential data, such as in natural language processing, serves various purposes, including sentence-level classification (e.g., sentiment analysis), next-word prediction, and token-level classification.
- Accurate modeling of sequential dependencies is crucial as word order and context greatly affect the meaning of text.
- Recurrent Neural Networks (RNNs) are powerful for capturing these dependencies.

Basic RNN models and applications

- RNNs comprise input, hidden, and output units, ideal for tasks like next-word prediction.
- Input units represent words, and hidden units connect to current and previous time stamps.
- RNNs use vector representations for input, hidden states, and output with one-hot encoding for words.
- The output unit predicts the next word based on conditional probabilities, choosing the word with the highest probability as the prediction.

Basic RNN models and applications

- In an RNN model, the input unit connects to a hidden unit, the hidden unit connects to the hidden unit at the next time stamp, and optionally, the hidden unit can connect to an output unit.
- Weight matrices U , W , and V , along with bias vectors a and b , describe the relationships between different unit types.
- The RNN model is formally defined with equations (10.19) and (10.20), with 'T' representing the input sequence length.

Basic RNN models and applications

- The activation functions for hidden states (f) and output units (g) are crucial components, often employing tanh and softmax functions, respectively.
- Training an RNN model involves incorporating the true target value (y^T) and the loss function (L).
- The equations (10.19) and (10.20) describe how hidden states and outputs are computed using linear transformations and activation functions.

Basic RNN models and applications

- RNNs differ from feed-forward neural networks as they maintain recurrent connections and parameter sharing through the same weight matrices U , W , and the same bias vector a across different time stamps.
- RNNs capture long-term dependencies in sequential data, as opposed to 1-D CNNs which are more localized.
- In a general next-word prediction setting, output units ($\hat{y}^t$) are introduced, connecting to each hidden state h_t , enabling the prediction of every next word in the sequence.

Basic RNN models and applications

- Output units are compared to the true target values to measure disagreement, and the RNN model is trained by minimizing the total loss L .
- RNN models like those in Fig. 10.29 and Fig. 10.30 have applications in sentence-level and token-level prediction.
- There are different RNN types, including models for learning hidden representations, image captioning, and bi-directional RNNs.

Basic RNN models and applications

- Bi-directional RNNs introduce additional hidden states (pt) for considering information from both past and future words.
- Combining RNN models, such as for next-word prediction, enables more complex applications like machine translation (sequence-to-sequence learning or seq2seq).

Gated RNNs

- Addressing long-term dependencies is a significant challenge in recurrent neural networks (RNNs).
- RNNs are effective for tasks where the predicted output is closely related to the previous input, but they struggle when the dependency between input and output spans a long sequence.
- Deep RNNs, with many hidden states, can theoretically capture long-term dependencies, but they are challenging to train due to issues like gradient vanishing.

Gated RNNs

- Long Short-Term Memory (LSTM) models offer a solution to the long-term dependence problem in RNNs.
- LSTM introduces gates (forget gate, input gate, output gate) that control the flow of information within the cell state and the hidden state.
- Forget gate regulates how much old information from the previous cell state is retained.

Gated RNNs

- Input gate controls how much new information from the temporary cell state is used to update the cell state.
- Output gate determines how much information from the new cell state contributes to generating the new hidden state.
- LSTM and its variants, like Gated Recurrent Units (GRUs), have been successfully applied in real-world applications such as machine translation, speech recognition, image captioning, and question answering.

Other techniques for addressing long-term dependence

- Techniques for addressing long-term dependence in RNNs include gated RNNs, shortcut connections, longer connections, and leaky units.
- An alternative strategy is the attention mechanism, which augments hidden states with context vectors to pass relevant information across longer gaps.
- To obtain context vectors, alignment vectors are used to measure the relevance between hidden states of the source and destination languages.

Other techniques for addressing long-term dependence

- The alignment vector can be calculated using various scoring functions, such as dot product or bilinear form.
- The attention mechanism effectively addresses long-term dependence and has been instrumental in architectures like Transformers and BERT.
- Memory mechanisms, which involve managing external memory for data mining models, are also used to address long-term dependence, with examples like neural Turing machines and memory networks.

Graph neural networks

- Graph data consists of nodes (vertices) connected by edges and is prevalent in various applications like social networks, power grids, financial transaction networks, and bioinformatics.
- Graph neural networks (GNNs) aim to transform graph data into multidimensional vectors, representing each node as a vector in a high-dimensional space.
- Using these vector representations, GNNs enable various tasks like churn prediction, community detection, fraud detection, failure prediction, recommender systems, and more.
- Graph data is turned into vectors through the application of GNNs, particularly graph convolutional networks (GCNs) and other types of GNNs.

Basic concepts

- A graph with n nodes can be mathematically represented using an adjacency matrix A of size $n \times n$, where the entries are 1s for connected nodes and 0s for unconnected nodes.
- Nodes and edges in a graph may have attributes, which can be represented using a node (or edge) attribute matrix X , where rows represent nodes (or edges), columns represent attributes, and entries represent attribute values.
- Grids (e.g., images) and sequences (e.g., text) can be treated as special types of graphs, with nodes and edges representing pixels or words and attributes like RGB colors or word content.

Basic concepts

- GNNs aim to map nodes to low-dimensional vectors (node embeddings) using neural networks. For example, in a social network, users can be mapped to 2-D vectors to capture user characteristics.
- These node embeddings can be used in downstream applications like classification, clustering, outlier detection, and link prediction, making GNNs a valuable technique for graph data analysis.

Graph convolutional networks

- GCNs (Graph Convolutional Networks) are effective models for graph data.
- They operate on input graphs represented by an adjacency matrix A and node attribute matrix X .
- GCNs use weight matrices W_l (for each layer l) to produce node embedding matrices Z_l , capturing node representations.
- The algorithm includes preprocessing and initialization steps to prepare the input graph, including adding self-edges, calculating degree matrices, and normalizing the adjacency matrix.

Graph convolutional networks

- GCNs produce node embeddings layer-by-layer, involving weighted aggregation, linear transformations, and nonlinear activations.
- The main difference from traditional neural networks is the propagation step (Step 6) that handles neighborhood information in graphs.
- GCNs can be used for various tasks such as classification, clustering, and outlier detection in graph data.
- Weight matrices W_l can be learned using backpropagation if class labels are available for a subset of nodes in the graph.

Other types of GNNs

- GraphSAGE (Graph SAmple and aggreGatE) is a type of GNN model and a generalization of GCNs (Graph Convolutional Networks).
- In GraphSAGE, the main difference from GCNs lies in the aggregation function used in Step 4.
- GraphSAGE supports various aggregation methods, including pooling aggregator, LSTM-based aggregation, and more.
- Another distinction is in Step 5, where GraphSAGE employs a separate weight matrix U_l for the current embedding h_{l-1} .

Other types of GNNs

- Various other types of GNNs have been developed, including propagation-based spatial graph convolutional networks and Graph Attention Network (GAT).
- Many GNNs and their variants are trained in a supervised fashion, requiring labeled training data.
- Unsupervised methods like LINE, DeepWalk, and node2vec aim to learn node embeddings without labeled data.

Other types of GNNs

- GNNs focus on learning node embeddings that capture topological information from the input graph, while knowledge graph embedding (e.g., TransE) considers node and edge attributes for richer semantics.
- GNNs often have shared model parameters among nodes, making them inductive, capable of generating embeddings for unseen nodes.
- Research challenges in GNNs include handling deeper architectures without oversmoothing, effective subgraph embedding, and developing models for multisourced graphs (Multi-GNNs).

Summary

- A neural network involves units with linear weighted sums and nonlinear activation functions.
- Deep neural networks have multiple layers and can handle complex functions.
- Backpropagation uses gradient descent to adjust model parameters during training.
- Challenges include optimization (finding cost minima) and generalization (performance on new data).

Summary

- Gradient vanishing issues occur when gradients approach zero during training.
- Adaptive learning rates, dropout, and pretraining help enhance deep network training.
- Cross-entropy is a common loss function for classification tasks.
- Convolutional neural networks (CNNs) excel with grid-like data (e.g., images).

Summary

- Recurrent neural networks (RNNs) are ideal for sequential data and tackle long-term dependence using LSTM and attention mechanisms.
- Graph neural networks (GNNs) map graph nodes to low-dimensional vectors, with graph convolutional networks (GCNs) being an effective type.